

REQ1: The Life in the Valley

A. Creature Production

Design 1: In-class reproduction logic aka Implement offspring spawning directly in SpiritGoat class.

Pros	Cons
Straightforward: no additional classes required.	Reproduction logic intermingles with movement, reducing clarity and testability.
	Modifications to reproduction triggers require changing SpiritGoat, breaking Open/Closed Principle

Design 2: Producible interface with ProduceBehaviour

Pros	Cons
Encapsulates reproduction in ProduceBehaviour, adhering to Single Responsibility.	Introduces interface and behaviour classes, adding initial complexity.
Facilitates extension: new producers need no changes to SpiritGoat.	

B. Egg Implementation

Design 1: Creating specific egg classes

Pros	Cons
Avoids the "fat interface" problem, i.e., inheriting too many unnecessary methods	Increased code duplication; shared functionalities must be re-implemented in each egg class
Avoids unnecessary constraints that inheritance may introduce, such as the hatching mechanism	Difficult to implement unified egg-related functionalities, such as a common hatching mechanism
	Difficult to manage all egg types uniformly, e.g., cannot use polymorphism effectively

Design 2: Produce Different Eggs via Passed-in Parameters

Pros	Cons
Uses the Condition interface to abstract and support multiple hatching conditions.	Memory usage may grow with many Egg instances.
Enables adding new hatching conditions and effects without changing existing code.	The number of parameters passing through Egg instance is long.
Promotes code reuse by configuring Egg instances with different parameters.	

C. Egg Consumption Mechanism

Design 1: Generic ConsumableItem base to handle eating effects.

Pros	Cons
Reuses consumption logic for all edible items.	May overgeneralize effects, forcing items to fit a uniform model.
Keeps effect application consistent.	Unique item effects can complicate the base class.

Design 2: Eatable interface and ConsumptionEffect composite.

Pros	Cons
Provides a unified abstraction for edible behaviour via the Eatable interface	It can be harder to trace and debug the effect chain.
Uses Composite Pattern List<Effect> to combine multiple consumption effects.	Composite pattern may add unnecessary complexity for simple cases.

D. AdjacentCapabilityChecker Implementation

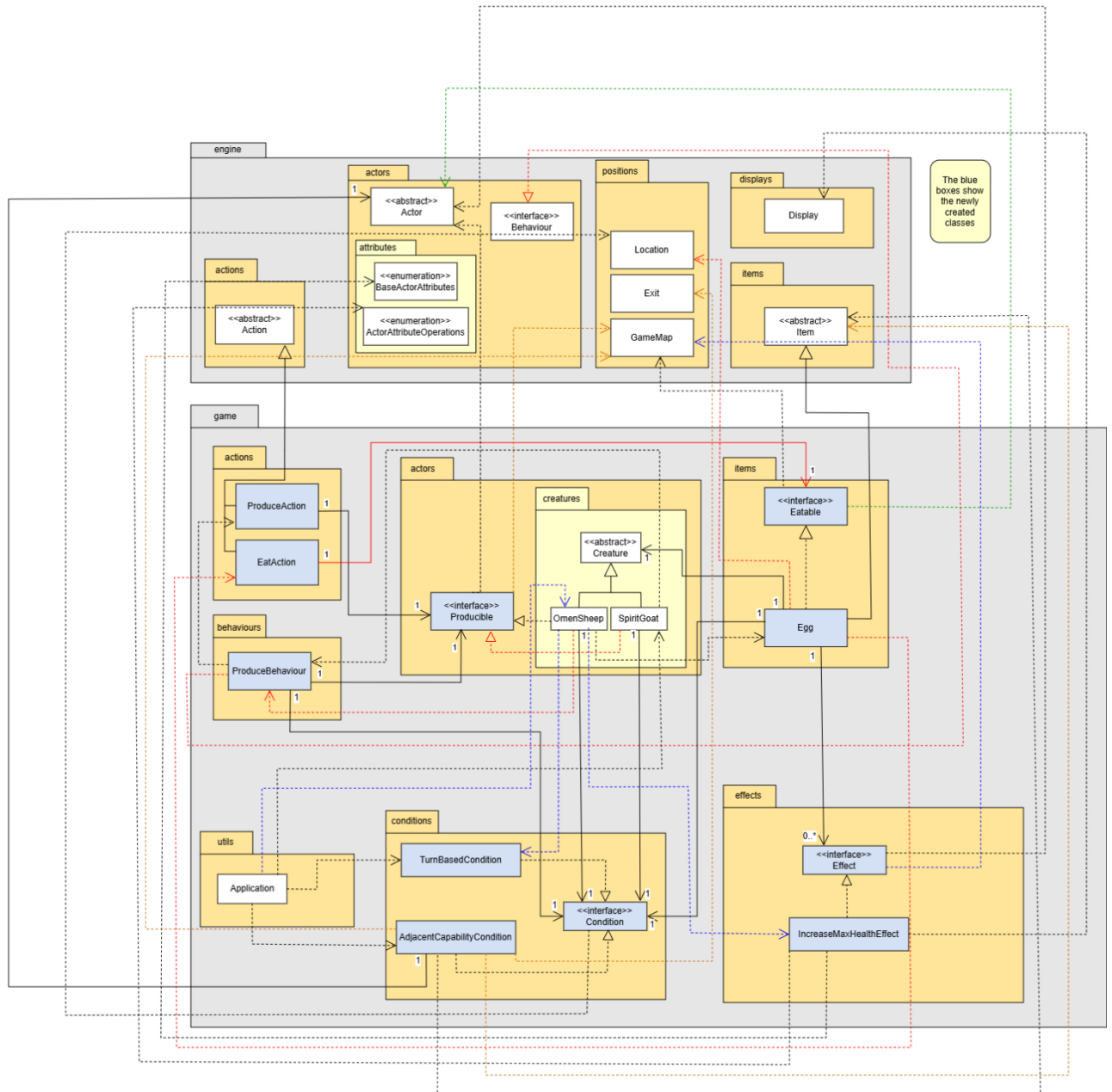
Design 1: Add a Condition (AdjacentCapabilityCondition) class.

Pros	Cons
Centralized code, easy to locate	Change in checking logic require additional classes.
Reduce code duplication and promotes DRY	
Able to adapt to different capabilities/enumeration in the future such as POISONED.	

Design 2: Embed checking logic directly in methods Implement adjacency checking logic directly in methods that require it.

Pros	Cons
Simplifies immediate implementation	Violates DRY principle through code duplication
No additional classes to manage	Changes to checking logic require updates in multiple places
Direct access to local context	Reduced testability as logic is scattered

REQ1 - UML Diagram



The above diagram shows the final design of Req1 in which we chose Design 2 for **Creature Production, Egg Implementation and Consumption Mechanism** whereas we use Design 1 for **AdjacentCapabilityChecker** implementation.

The architecture we've developed strictly adheres to SOLID principles, which provides significant benefits for maintainability and extensibility. For instance, the Single Responsibility Principle (**SRP**) is demonstrated through our clear separation of concerns: ProduceBehaviour and ProduceAction exclusively manages produce behaviours and EatAction handles consumption effects.

This separation prevents any single class from becoming a "god class" with too many responsibilities, making the codebase easier to understand, test, and maintain.

Consider a hypothetical scenario where we need to modify how creatures reproduce. In a non-SRP design, we would need to modify the SpiritGoat class directly, potentially introducing bugs in unrelated functionality. Our implementation isolates reproduction logic in ProduceBehaviour and ProduceAction, allowing targeted changes without disturbing movement or rendering code.

The Open/Closed Principle (**OCP**) is respected through our behavioural injection approach with the Producible and Eatable interfaces. When a new creature type needs to be added, we can simply implement these interfaces without modifying existing framework code. For example, if requirements change to include a new "PhoenixBird" that reproduces differently, we can make it implement the Producible interface and implement its own produce method without changing a single line of code in the core framework.

The Interface Segregation Principle (**ISP**) is reflected in our carefully scoped interfaces. For example, Producible exposes only methods related to production capabilities, while Eatable focuses exclusively on consumption behaviours. This prevents the "fat interface" problem where implementers are forced to provide irrelevant functionality.

The Dependency Inversion Principle (**DIP**) is evident in how high-level modules like the game engine and action scheduler depend on abstractions (ProduceBehaviour, EatAction) rather than concrete implementations. This inversion allows the engine to remain agnostic about specific producers or consumables, enhancing system flexibility and testability. For instance, the game engine doesn't need to know whether a SpiritGoat or OmenSheep is producing offspring as it simply works with the Producible interface. Instead of tying the system to specific subclasses like a parameterized Egg class, it encapsulates shared lifecycle logic. Specific behaviours are injected via parameters or strategies, keeping the system flexible, decoupled, and easy to extend without altering core logic, enhancing maintainability and scalability.

We've also prioritized the Don't Repeat Yourself (**DRY**) principle throughout our design. By centralizing adjacency checking in the AdjacentCapabilityChecker utility class rather than embedding this logic in various methods, we've eliminated potential duplication and ensured consistent behaviour. Instead of separate subclasses like OmenSheepEgg or GoldenBeetleEgg, a single parameterized Egg class handles common lifecycle functions. Specific behaviours are

set via instantiation parameters, reducing code duplication and improving flexibility by enabling new egg types without new subclasses.

From a connascence perspective, our design carefully manages various coupling types to enhance maintainability. It addresses Connascence of Name through consistent method naming conventions across interfaces and implementation classes. Connascence of Algorithm and Type is managed through interfaces that enforce specific method signatures and return types, ensuring uniform interaction patterns. Connascence of Timing and Value is handled through well-parameterized configurations, making behaviour execution predictable.

If new requirements emerge, such as adding new creature types or changing reproduction mechanics, our design ensures minimal impact on existing systems. For example, if business requirements change to make reproduction dependent on time of day, we would only need pass in the produce condition to the creature such as OmenSheep and SpiritGoat which makes it easy extensible.

Our design comprehensively addresses all requirements while establishing a robust foundation that anticipates future changes. The architecture provides clear extension points for new features while maintaining a cohesive overall structure, balancing immediate implementation needs with long-term maintainability.

